UTN.BA

UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Algoritmos y Estructuras de Datos

2026

Docente: Pablo Mendez

Trabajo Práctico Grupal Nº1

Curso K1153 Fecha de entrega : 04/11/2026		
Legajo	Nombre Apellido	Correo
222710-1	Alejandro Silva	asilvacastro@frba.utn.edu.ar
USUARIO GITHUB : AleSilvaUTN		
NOTA:		
FIRMA DOCENTE		
Observaciones:		

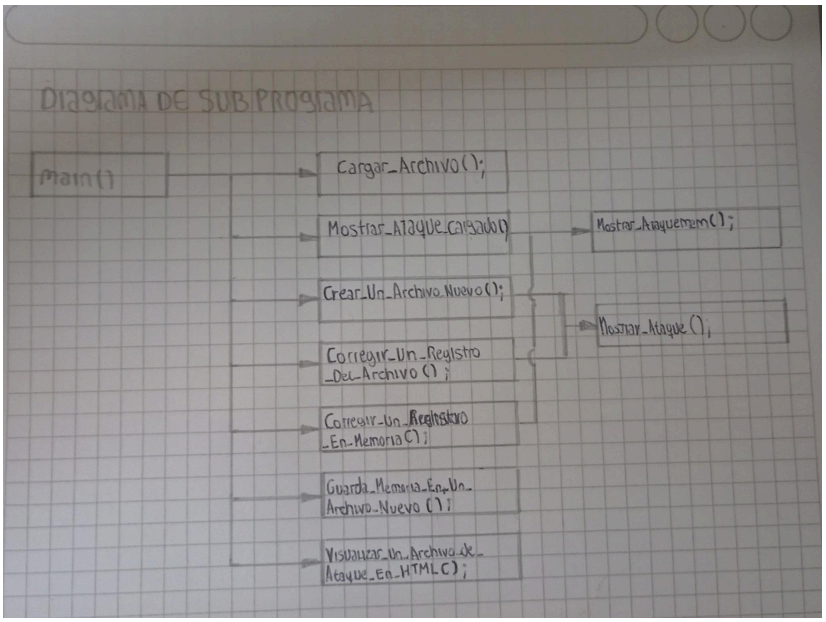
INTRODUCCIÓN Y OBJETIVOS

El objetivo de este trabajo práctico se basa en aplicar los conocimientos aprendidos en relación a temas del primer cuatrimestre combinado con el tema de ARCHIVOS el cual busca averiguar cómo resolver 7 operaciones fundamentales.
Estas operaciones son :

1. Cargar archivo de ataque en memoria.
2. Mostrar ataque cargado.
3. Crear un archivo de ataque nuevo.
4. Corregir un registro del archivo.
5. Corregir un registro en memoria.
6. Guardar memoria en un archivo nuevo.
7. Visualizar un archivo de ataque en html.

Estas operaciones vienen dadas bajo ciertas reglas escritas en el trabajo práctico pero también se debe realizar bajo criterios y suposiciones del estudiante, viendo qué método o lógica servirá a la hora de dar una solución a cada operación.
Estas operaciones estarán dentro de la función MAIN(); donde mediante un post condicionado DO WHILE y dentro una sentencia selectiva SWITCH(); se irá seleccionando lo que necesite el usuario.

Cómo funcionamiento de cada operación hice un diagrama de bloque de subprograma



Este programa creado está pensado en que el usuario ya conoce desde antes de iniciar el programa la trayectoria, no se puede corregir de manera perfecta por pura intuición, siempre visualizando la tabla 200x200.

Por último quiero aclarar que anteriormente yo tenia declarado la memoria 200x200 del dron y un bool 200x200 (usado para ver si la coordenada ocupa datos) en el main() que son los

datos importantes para usarse en las demás operaciones pero por revisión de una IA tanto CHATGPT como CLAUDE, me indicaba de un grave stack overflow por parte de estas declaraciones que mencione antes y también por las que declare en la operación 1.
Según menciona esto podría ocasionar un cierre forzoso del programa, en mi caso ejecutaba normal pero aun asi decidi realizar las declaraciones en global, durante el informe verá que yo menciono en cada subprograma los parámetros declarados como la memoria del dron pero como puede ver es innecesario ese parámetro.
Esto es así de forma intencional ya que considero que no es necesario declararlo en global, así que simuló que estas declaraciones están acorde a donde supuse antes.

La resolución de todos los problemas están dadas por mi.

OPERACIÓN 1 => CARGAR ARCHIVO;

Mi solución en este subprograma consiste primero en analizar las variables tipo bool de OrdenArchivo de cada coordenada que tenga el archivo, este propósito se debe a que busco que primero haya una trayectoria definida por el usuario, que tenga un inicio y fin, y que también no haya coordenadas repetidas. Luego de que tengamos una trayectoria definida busco determinar ahora la trayectoria a detalle, si las coordenadas están en diagonal o si no son continuas esto debido a los criterios que establece el profesor por último si esta todo bien se guarda a la memoria del dron.

Bajo mi criterio, con respecto a soltar granada, yo considero que no tiene sentido soltar la granada 1 o 2 más de una vez, el dron tendrá una capacidad de 2 solamente.
También considero que en una coordenada se pueden soltar las 2 granadas a la vez con el fin de hacer más daño a la zona.
Esto último también aplica cuando se desea finalizar, se puede realizar mayor daño si por ejemplo quiero finalizar el recorrido con un ataque kamikaze y también soltar las 2 granadas.

PLANTEO:
Este subprograma lo tuve que definir como un tipo bool (bool cargar_archivo) ya que considero necesario para las operaciones:
MOSTRAR ATAQUE CARGADO **(2)** – CORREGIR UN REGISTRO EN MEMORIA **(5)** – GUARDAR MEMORIA EN UN ARCHIVO NUEVO **(6)**y VISUALIZAR EN UN ARCHIVO DE ATAQUE HTML **(7)**
Esto debido a que antes de proceder con aquellos subprogramas, el dron debe tener memoria, este subprograma deberá retornar a TRUE o FALSE dependiendo si verifica las condiciones que he establecido.
En el main() declare un aux **CargarAUX** que simula un primer cargado ya que si carga un archivo a la memoria por primera vez entonces el dron permanece con esos datos hasta finalizar el programa, este auxiliar se usará para las demás operaciones antes de proseguir a la operación que se desea.

Fuera del main declare de manera global, **memoria_temporal[xmax][ymax] , bool coord_ocupada[xmax][ymax]** estos 2 solamente para esta operación que busca simular que la trayectoria es correcta y las coordenadas de la trayectoria también.

Por otra parte este subprograma tendrá estos 3 parámetros , **memoria[][], coordenada_ocupada[][]** y **char ruta[256]** que es la ruta donde se debe encontrar el archivo.

Dentro del subprograma plantee esta secuencia

1. Abro el archivo y con la ruta ingresada en el main se comprueba si encontró o no el archivo
2. Realizo limpieza en la **memoria_Temporal[][]** y **coord_ocupada[][]** para su uso más adelante, esto se debe a que al ser globales si realizó cambios en ellas mantendrán esos datos guardados.

3. Con un **While** recorro el archivo buscando Despegues, Aterrizajes, Ataque Kamikaze, Soltar granada 1 y 2 todo en **TRUE**
   Al ser true, anteriormente declaro variables de tipo entera CANT_DE_DESPEGUES=0,CANT_DE_FINALIZACIONES=0, CANT_DE_S1 = 0, CANTA_DE_S2=0, el cual busca aumentar uno en uno para cuando encuentre un TRUE.
   Cuando detecto un despegue, variables de tipo entera dx y dy tendran las coordenadas de inicio de la trayectoria.
4. Regreso con **Fseek** al punto inicial y el **Orden memoria_temporal[][]** va a guardar cada registro de cada coordenada X e Y del archivo con datos.x y datos.y.
   También **coord_ocupada[][ ]** guarda registro de cada coordenada ingresada indicando si hay una coordenada repetida, esto ya que entonces en esas coordenadas se van a modificar 2 veces y como condición que plantea el TP, no debe pasar eso aun asi no se detendrá el subprograma, al marcar error un bool **errorepe** pasará a true que determinara el fin de la secuencia 4.
   Por último en caso de no repetirse, una variable entera **recorrido = 0** aumentará uno en uno y esto se usará en un **FOR** para recorrer la trayectoria más adelante**.**

5. Con la determinación de la secuencia 2 y 3 con condicionales **IF** se va a determinar si hay errores para ello declaro variables de tipo bool similar a cant_de…, pero ahora en bool e inicializadas en false.
   En el TP se menciona que en la memoria solo debe existir un solo despegue y un tipo de finalización ya sea Aterrizaje o Ataque Kamikaze pero que sea solo uno, en cuanto a soltar granada yo ya mencione mi consideración.
   Entonces los IF determinarán si esto cumple o no esto gracias a la secuencia 2. En caso de no cumplir el bool pasará de **FALSE** a **TRUE**. y con el último condicional IF determinara que al menos un bool sea **TRUE** para finalizar el subprograma retornando **FALSE**.

```
//marco errores que se detecta rapido
if(cantidad_de_despejes > 1)
{
if(cantidad_de_despejes == 0)
{
if(cantidad_de_finalizaciones > 1)
{
if(cantidad_de_finalizaciones == 0)
{
if(veces_que_se_suelta_granada_1 >1 )
{
if(veces_que_se_suelta_granada_2 >1 )
{
if(errorDes || errorfina || errorG1 || errorG2 || errorRep)
{
```

Por último en caso de que un condicional se cumpla menos el último dentro mediante un FOR mostrará las coordenadas para corregir.

6. En esta parte se busca analizar las coordenadas, antes de eso declaró variables de tipo entera x,y,rX,rY y declaro una variable tipo bool finalizacion = false; también gracias a dx y dy entonces asignó x = dx , y = dy
7. Dentro del for(int i = 0; i < recorrido; i++) , habrá condicionales IF que buscan cumplir con el manejo de coordenadas.
   Primeramente

```
if(memoria_temporal[x][y].despegue == true)
{
    if(memoria_temporal[x][y].soltarGranada1 || memoria_temporal[x][y].soltarGranada2 || memoria_temporal[x][y].ataqueKamikaze)
    {
        cout<<"Error coordenadas (x,y) ==> ("<<x<<","<<y<<")"<<endl
        << "Hubo ataque en el despegue"<<endl;
        fclose(archivo);
        return false;
    }
}
```

Esta condición describe que cuando hay un despegue, no debe tener ningún tipo de ataque ni finalización, solamente siguientes x e y.
2do

```
if(memoria_temporal[x][y].ataqueKamikaze || memoria_temporal[x][y].aterrizaje)
{
        // verifico si se finaliza con ambos meotodos
        if(memoria_temporal[x][y].ataqueKamikaze && memoria_temporal[x][y].aterrizaje)
        {
            cout<<"Error coordenadas (x,y)==> ("<<x<<","<<y<<")"<<endl;
            cout<<"Intento de doble finalizacion"<<endl;
            fclose(archivo);
            return false;
        }
        // verifico si hay un siguente movimiento
        if(memoria_temporal[x][y].siguientex > 0 || memoria_temporal[x][y].siguientey > 0)
        {
            cout<<"Error coordenadas(x,y)==> ("<<x<<","<<y<<")"<<endl
            <<"Al intentar finalizar el dato de la cooordenada marca una coordenada siguente mayor a (0,0)"<<end
        fclose(archivo);
        return false;
        }else
        {
            i = recorrido+1; // se fuerza el cierre del for
            finalizacion = true;
            }
}else
```

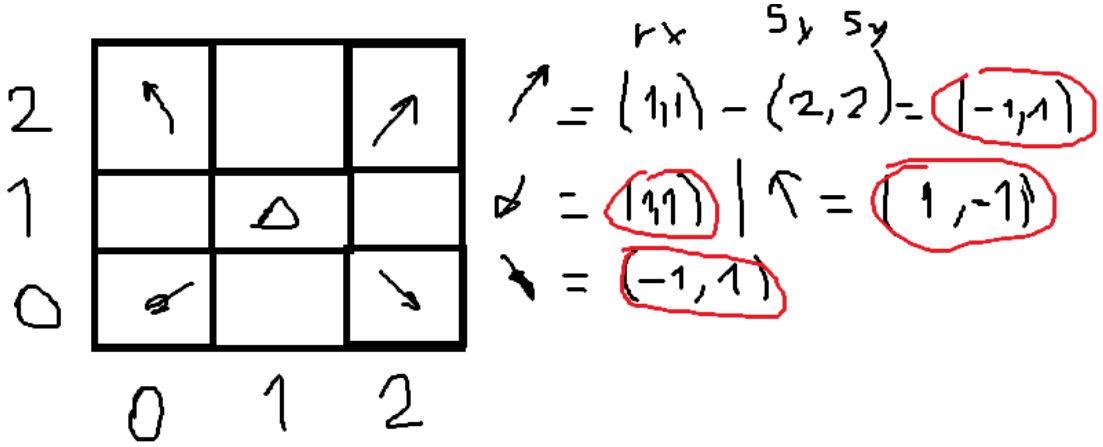
Aca con este if consulto si se termina el recorrido
En caso de que ataque **Kamikaze** o **Aterrizaje** sean **TRUE**, entonces dentro hago otro **IF** verifica si la finalización proviene de ambas, en caso de ser cierto entonces sería un error.

En caso de que no sea entonces consultando por otro **IF** si dentro de esas coordenadas se va a “realizar” otro movimiento aunque no haya un registro que confirme, quizás esas coordenadas este vacío pero de todas formas se debe considerar de que si hay un movimiento siguiente entonces hay un error.
Por último , si entonces no hay movimientos finaliza el **FOR**, forzando el cierre para proseguir fuera del **FOR** y también a finalización se le asigna true.
3ro

```
}else
{
    // Se comprueba los movimientos diagonales
    rX=x - memoria_temporal[x][y].siguientex;
    rY=y - memoria_temporal[x][y].siguientey;
    if((rX == -1 && rY == -1)||(rX == -1  && rY == 1 )||(rX == 1 && rY == 1 )||(rX == 1 && rY == -1 )||(rX ==0 && rY==0)
    {
        cout<<"Error coordenada (x,y) ==> ("<<x<<","<<y<<")"<<endl
        <<"Movimiento diagonal"<<endl
        <<"En caso de no encontrar las coordenadas pruebe con la coordenada anterior ("<<dx<<","<<dy<<")"<<endl;
    fclose(archivo);
     return false;
    }
    // Se comprueba los movimientos que no sean continuos
    if((rX > 1 || rX < -1) || (rY > 1 || rY < -1))
    {
        cout<<"Error coordenada (x,y) ==> ("<<x<<","<<y<<")"<<endl
        <<"Movimiento no continuo"<<endl
        <<"En caso de no encontrar las coordenadas pruebe con la coordenada anterior ("<<dx<<","<<dy<<")"<<endl;
    fclose(archivo);
    return false;
    }
    // uso rX y rY como auxiliar para las siguentes coordenadsa
    // uso dx y dy como valores anteriores por si las siguientes coordenadas no existen en el registro
    dx = x;
    dy = y;
    rX = memoria_temporal[x][y].siguientex;
    rY = memoria_temporal[x][y].siguientey;
    x = rX;
    y = rY;

}
```

Este **ELSE** sucede al saber que no termina el recorrido, por lo que verificamos los movimientos continuos, no diagonales.
Antes de proceder primero emplee un cálculo simple el cual se usarán rX y rY
su resultado en caso de ser continuo para rX será 1 o 0 y lo mismo para rY
Con el primer IF verificó diagonales bajo este criterio



y con el otro IF si las variables rX y rY son mayores a 1 o menores a 0, no cumple con continuidad.
Por último, como se que no hubo error entonces usó rX y rY como aux para las siguientes coordenadas y dx dy como aux para las coordenadas anteriores.
El propósito de dx dy es que como la memoria toma los valores de las coordenadas siguientes , estas coordenadas no aseguran que este en el registro, es por ello en que si hay un error de continuidad o diagonal y me tira una coordenada a corregir

que quizás no exista en el registro, entonces tambien le tendre que pasar las coordenadas anteriores que esos si existen en el registro.

8. Como ultimo pregunto si finalizacion es false, si es false entonces hay un error de ciclo, que quiere decir, supongamos esta trayectoria (2,2) => (2,3) =>(2,2) y afuera (3,3) el cual este finaliza pero no pertenece a la trayectoria, esta trayectoria es infinita y verdadera ya que es continua así que “i” va aumentando hasta salir del FOR como si hubiera pasado correctamente pero no es así.
   Con este IF verificar que haya pasado por el último punto de la trayectoria.

```
if(finalizacion == false)
{
    cout<<"Se intento salir del FOR mediante un ciclo"<<endl
    <<"Corregir coordenadas ("<<x<<","<<y<<")"<<endl;
    fclose(archivo);
    return false;
}
```

9. En caso de que la memoria del dron ya tenga contenido por un anterior ataque cargado, entonces tendré que hacer limpieza con un FOR, lo mismo con **coordenada_ocupada[][]** que sirven para otras operaciones.
10. Vuelvo al archivo y uso FSEEK para tener que apuntar al inicio. y con WHILE realizó el pasaje de datos con el registro, también podría haber usado la memoria temporal pero con el registro se hace más rápido el proceso aunque fuese imprescindible para nosotros. Para finalizar también paso los datos del **bool coord_ocupada[][]** temporal a la **coordenada_ocupada[][]** limpia.
11. Por último retorno TRUE

PROBLEMA QUE NOTE

Si bien el subprograma funciona, hay un pequeño error

-	0	1	2	3	4	5
0						
1						
2			D			
3			Sg2 |			
4			|	|		
5			|			
6			A			
7						

En esta ocasión cree un archivo de ataque que empieza de 2,2 y termina en 2,6 siendo una trayectoria recta pero también coloque movimiento en 3,3 a 3,4 siendo éstos independientes de la trayectoria empleada pero cómo vemos que el **recorrido** es de 7 al llegar a Aterrizaje 2,6 deja de leer lo demás, como la secuencia 4 evitar que haya más despegues y finalizaciones entre otros, entonces dejaría pasar solamente movimientos fantasmas. Es por ello que la solución que vi a esto es usando la operación 5 el cual viendo la tabla html e identificando las coordenadas, puedo hacer que 3,3 y 3,4 sus movimientos sean a (0,0)

```
Ingrese coordenadas (x,y) ==>
3
3
===DATOS==

Depegue              ==> 0
Aterrizaje           ==> 0
Soltar granada       ==> 0
Soltar granada 2     ==> 0
Ataque Kamikaze      ==> 0
Esperar              ==> 0
Siguente coordenadas ==> (3,4)
Presiona cualquier tecla para continu
Que desea cambiar
Depegue          (1)
Aterrizaje       (2)
Soltar granada   (3)
Soltar granada 2 (4)
Ataque Kamikaze  (5)
Esperar          (6)
Siguiente x e y  (7)
Finalizar        (8)
COORD Sx Sy actual => (3,4)
Coordenada a reemplazar (x,y) =>
0
0
```

```
Corregir otras coordenadas? True (T) False (F)
Ingrese coordenadas (x,y) ==>
3
4
===DATOS==

Depegue              ==> 0
Aterrizaje           ==> 0
Soltar granada       ==> 0
Soltar granada 2     ==> 0
Ataque Kamikaze      ==> 0
Esperar              ==> 0
Siguente coordenadas ==> (3,5)
Presiona cualquier tecla para continuar
Que desea cambiar
Depegue          (1)
Aterrizaje       (2)
Soltar granada   (3)
Soltar granada 2 (4)
Ataque Kamikaze  (5)
Esperar          (6)
Siguiente x e y  (7)
Finalizar        (8)
COORD Sx Sy actual => (3,5)
Coordenada a reemplazar (x,y) =>
0
0
```

-	0	1	2	3	4	5
0						
1						
2			D			
3			Sg2			
4			|			
5			|			
6			A			
7						

OPERACIÓN 2 => MOSTRAR ATAQUE CARGADO

Para esta operación mi solución se basa en buscar el inicio de la trayectoria luego con un WHILE realizó el muestreo de datos y uso como ayuda el getch(); para no ver los registros de corrido.

PLANTEO:
Para esta operación primero debo verificar si en el main() => **CargarAux** es TRUE esto para verificar que el dron tenga datos en la memoria.
Como parámetro para el subprograma solamente necesitare de la memoria[][];
Dentro del subprograma tipo VOID declaró enteros x,y,aux1 y aux2 y bool encontrado y fin registro ambos en FALSE
Primero realizó una búsqueda a las coordenadas a despegue == TRUE que es donde empieza el recorrido.
Después con un WHILE(!fin_registro) se realiza el muestreo de datos, dentro del while hay un IF que preguntara si .aterrizaje o .ataque kamikaze está en true dado que sea cierto entonces el fin_registro pasa a ser true y muestra en pantalla el último registro.
Algo a aclarar es que hice otro subprograma llamado **mostrar ataque mem** que tiene como objetivo mostrar el ataque cargado en memoria, esto lo hice ya que puedo usarlo en la **operación 5** ya que al corregir **coordenadas de la memoria cargada** debo ver que datos tiene la coordenada en caso de existir.

```
void mostrar_ataquemem(Orden memoria[Xmax][Ymax],int x, int y)
{
 cout<<"===DATOS=="<<endl
        <<endl<<"Depegue              ==> "<<memoria[x][y].despegue
        <<endl<<"Aterrizaje           ==> "<<memoria[x][y].aterrizaje
        <<endl<<"Soltar granada       ==> "<<memoria[x][y].soltarGranada1
        <<endl<<"Soltar granada 2     ==> "<<memoria[x][y].soltarGranada2
        <<endl<<"Ataque Kamikaze      ==> "<<memoria[x][y].ataqueKamikaze
        <<endl<<"Esperar              ==> "<<memoria[x][y].espera
        <<endl<<"Siguente coordenadas ==> ("<<memoria[x][y].siguientex<<","<<memoria[x][y].siguientey<<")"<<endl;
}
```

También lo realice para registros donde se usará para la operación 3 y 4 .

```
void mostrar_ataque(OrdenArchivo a)
{
        cout<<"===DATOS==="<<endl
        <<endl<<"Depegue              ==> "<<a.despegue
        <<endl<<"Aterrizaje           ==> "<<a.aterrizaje
        <<endl<<"Soltar granada       ==> "<<a.soltarGranada1
        <<endl<<"Soltar granada 2     ==> "<<a.soltarGranada2
        <<endl<<"Ataque Kamikaze      ==> "<<a.ataqueKamikaze
        <<endl<<"Esperar              ==> "<<a.espera
        <<endl<<"Siguente coordenadas ==> ("<<a.siguientex<<","<<a.siguientey<<")"
        <<endl<<"Presiona cualquier tecla para continuar"<<endl;
        getch();
        return;
}
```

PROBLEMA QUE NOTE.
Este problema es algo rebuscado ya que debe ser ocasionado de manera intencional.
Supongamos que al corregir la memoria, el bool que busca finalizar lo quiero cambiar a FALSE, entonces se entiende que la trayectoria tiene un inicio, pero no un final.
En ese caso el WHILE que muestra los datos por pantalla termina siendo un bucle ya que no encontrará el fin.

OPERACIÓN 3 => CREAR ARCHIVO NUEVO

Para esta operación necesitare de Archivo que se abra con **wb** para escribir junto con la ruta.
También como parámetro para el subprograma necesitare de **char ruta[256];**

Mi solución para crear este subprograma es el ingreso de coordenadas, controlo que no pasen de rango establecido, muestro los datos de esas coordenadas y el menú de operaciones para hacer, selecciono lo que necesito y al finalizar consulto si pasar a otras coordenadas o pasar a las coordenadas siguientes ingresadas.

PLANTEO:
Para ello primero preguntó con IF si el archivo se abre

Después procedo a meter datos esto mediante un **DO WHILE**
Dentro de ahí primero ingreso las coordenadas que quiero ingresar datos pero con cierta conciencia de que no debe pasar el rango, un error muy evidente, en ese caso me voy del subprograma.
Sabiendo eso y pasar de la condición de rango se debe entrar a otro **DO WHILE** donde ahi uso el subprograma mostrar_ataque() , este mostrará los datos que estaré ingresando luego debo mostrar un menú de opciones que quiera hacer

```
do
    {
        mostrar_ataque(AtaqueNuevo);
        cout<<endl<<"COORDENADA ACUTAL      ==> ("<<AtaqueNuevo.x<<","<<AtaqueNuevo.y<<")"<<endl;
        cout<<"Que desea cambiar"
        <<endl<<"Depegue              (1)"
        <<endl<<"Aterrizaje           (2)"
        <<endl<<"Soltar granada       (3)"
        <<endl<<"Soltar granada 2     (4)"
        <<endl<<"Ataque Kamikaze      (5)"
        <<endl<<"Esperar              (6)"
        <<endl<<"Siguente coordenadas (7)"
        <<endl<<"Finalizar            (8)"<<endl;
       opcion = getch();
        no_terminar = true;
switch(opcion)
```

con un getch() que asigna a una variable declarada entero opción pasará a switch que buscara lo que se desea.
opción al querer ser asignado con getch() , opción va a adquirir el número ASCII del valor que elegimos , si typeo 1 opción adquiere 49.

```
{
    case 49:
    do
        {
            cout<<"despegue ==> True (T) o False (F) "<<endl;
            asignacion = getch();
            if(asignacion == 116 || asignacion == 84)     // t = 116 , T = 84
                AtaqueNuevo.despegue = true;
            if(asignacion == 102 || asignacion == 70)     // f = 102 , F = 70
                AtaqueNuevo.despegue = false;
        }while(asignacion != 116 && asignacion != 84 && asignacion != 102 && asignacion != 70);
    break;
    case 50:
    do
```

esto mismo aplica para cuando quiera asignar true o false.
Ahora si quiero terminar de editar el registro de esas coordenadas al salir del do while, tendré que mostrar si deseo terminar de ingresar coordenadas, esto lo hago con otro **DO WHILE** por si me equivoco al tipear otro valor, el objetivo del do while es que si nos equivocamos al presionar una tecla y no era lo que queríamos, el subprograma no castigue asi que con un do while evitamos eso.
En caso de querer seguir ingresando coordenadas yo inserte 2 cosas, si quiero ingresar coordenadas nuevas o la continuación de las coordenadas siguientes que coloque en las coordenadas anteriores es por ello que con lo último hay que tener cuidado ya que pueden ser (0,0) en caso de no haber ingresado coordenadas siguientes anteriormente.
Para las condiciones del while tuve que declarar
bool no_finalizar = true el cual este pasará a false cuando ya no quiera ingresar más coordenadas/
bool primer_ingreso= true esto para ingresar las primeras coordenadas y también cuando quiera ingresar coordenadas nuevas.
bool no_terminar = true, usado para cuando ya no quiera modificar el registro de ciertas coordenadas.

OPERACIÓN 4 => CORREGIR UN REGISTRO DEL ARCHIVO

Para este procedimiento tendré que hacer algo similar al anterior solo que al abrir el archivo debo leer y escribir **rb+**.

Mi solución en este procedimiento es: ingresar coordenadas, buscar la busqueda en el archivo, si encuentra debe mostrar por pantalla los datos de esas coordenadas y el menú de operaciones donde el usuario determinará qué querrá cambiar.

En este caso cada que quiera buscar coordenadas para corregir , uso **FSEEK** para empezar desde el inicio cada que quiera buscar coordenadas y con un **while** realizó la búsqueda de coordenadas junto con un IF que consultaré si encontró similitud.
En caso de que si haya encontrado ingreso a un **DO WHILE** con el objetivo de corregir el archivo, en esta parte el procedimiento es el mismo que el anterior punto solo que hay una diferencia **MODIFICAR X e Y**. Su propósito busca que cuando haya **coordenadas repetidas**, este pueda modificar esas coordenadas moviéndose a otro lado. Esto en mi opinión arregla el tema de coordenadas repetidas a menos que el archivo esté corrupto.
Por último necesitare de una variable tipo entero “i=0” que servirá como puntero y se usará a la hora de reescribir un registro

```
            case 57:
                fseek(archivo,i*sizeof(OrdenArchivo),SEEK_SET);
                fwrite(&registro,sizeof(OrdenArchivo),1,archivo);
                fflush(archivo);
                no_terminar = false;
            break;
        }

    }while(no_terminar);
```

uso fflush() ya que por fuentes de internet e IA tuve ciertos problemas y con la ayuda de esto puedo forzar el pasaje de datos al archivo.

OPERACIÓN 5 => Corregir un registro en memoria

Como parámetro para este considero la memoria[][] y **coordenada_ocupada[][]**
Mi solución en esta operación es similar al anterior punto solo que con pequeños cambios
PLANTEO:
Esta operación realiza una búsqueda mediante el bool **coordenada_ocupada[][]** el cual como sabemos guarda el dato de que existe la coordenada, esto se realiza mediante un IF el cual primeramente debo revisar el rango y luego verifico con el bool si existe la coordenada ocupada entonces con un **DO WHILE** realizó los cambios, similar al anterior solo que este utiliza la memoria de ataque cargado y no tiene **MODIFICAR X E Y**.

OPERACIÓN 6 => GUARDA MEMORIA EN UN ARCHIVO NUEVO

Mi solución para esta operación es buscar el inicio de la trayectoria y se realiza el pasaje de datos a un archivo.
Como parámetros necesitare de memoria[][] y char ruta[256]
PLANTEO:
Para poder operar con esta operación se debe comprobar si hay memoria cargada en el dron mediante el bool declarado en el main() => **CargarAUX .**
En caso de verificar entonces se tendrá que declarar dentro del subprograma una variable tipo **OrdenArchivo guardado**.
Antes de empezar tendré que verificar que el archivo se abra.
Despues tendre que realizar una búsqueda donde empieza la trayectoria, para eso debo encontrar memoria[i][j].despegue == true,
Si encontró entonces con un **DO WHILE** se hace el pasaje de datos de memoria[x][y] al **guardado** mediante asignaciones, para frenar el pasaje de datos al archivo se debe encontrar que finalice la memoria y eso es si **Aterrizaje y Ataque kamikaze está en TRUE** esto mediante un condicional IF.

```
if(memoria[x][y].aterrizaje == true || memoria[x][y].ataqueKamikaze == true) // ultimo guardado al finalizar
{
    guardado.x              = x;
    guardado.y              = y;
    guardado.ataqueKamikaze = memoria[x][y].ataqueKamikaze;
    guardado.despegue       = memoria[x][y].despegue;
    guardado.aterrizaje     = memoria[x][y].aterrizaje;
    guardado.soltarGranada1 = memoria[x][y].soltarGranada1;
    guardado.soltarGranada2 = memoria[x][y].soltarGranada2;
    guardado.espera         = memoria[x][y].espera;
    guardado.siguientex     = memoria[x][y].siguientex;
    guardado.siguientey     = memoria[x][y].siguientey;
    X                       = memoria[x][y].siguientex;
    Y                       = memoria[x][y].siguientey;
    fwrite(&guardado,sizeof(OrdenArchivo),1,archivo);
    fin = false; // pone en fin el do while
    fclose(archivo);
}else
```

Como problema se encuentra la misma situación que en la operación 2 al cambiar manualmente los datos de finalización con la operación 5. pero en este caso es más grave
Como esta operación trata de darnos un archivo, si entonces realizamos el bucle, el archivo se irá almacenando espacio con grandes tamaños es por eso que en esta operación tuve que corregirlo para ello declare antes una variable de tipo entero “k=0” y dentro del do while un condicional que pregunta si k >= (xmax*ymax) y también los rangos en caso de modificar eso también.
Por último uso fclose() para cerrar el archivo.

```
do
{
    if( (k >= (Xmax*Ymax)) || x < 0 || x >= Xmax || y < 0 || y >= Ymax)
    {
        cout<<"La trayectoria es invalida fuera de rango o ciclo sin fin"<<endl;
        fclose(archivo); // visto en internet
        return;
    }
```

Ahora necesito comprobar esto así que creo una trayectoria (2,2) (2,3)(2,4) y realizó los cambios primero creo el nombre de archivo que lo llamaré error.

Primero realizó qué pasaría si el aterrizaje es falso así que hago ese cambio en la operación 5 y luego ejecuto 6 con nombre error2.

```
Ingrese nombre de ruta para el archivo
error2
La trayectoria es invalida fuera de rango o ciclo sin fin
```

Ahora modificó las coordenadas siguientes fuera de rango y ejecuto 6 con nombre error3.

```
Ingrese nombre de ruta para el archivo
error3
La trayectoria es invalida fuera de rango o ciclo sin fin
```

OPERACIÓN 7 => VISUALIZAR UN ARCHIVO EN HTML

Para realizar este subprograma me ayudé del video de usted que se encuentra en youtube y la pequeña guía de HTML del anexo 2.
Con el código del anexo 2 se puede hacer la tabla 200x200 pero tiene un problema y es que no mostraba las coordenadas ni el recorrido del ataque cargado.
Lo que buscaba como solución es en la colocación de coordenadas numéricas (0,199) tanto vertical como horizontal , después colocar las coordenadas que abarquen datos de la memoria con el coordenada_ocupada[][]==true mediante un if y for.

PLANTEO:
Para ello primero me centré en la colocación de números en horizontal antes creo la fila con <tr> y luego con un for colocó celda por celda con <td>%d</td> y cierro </tr>

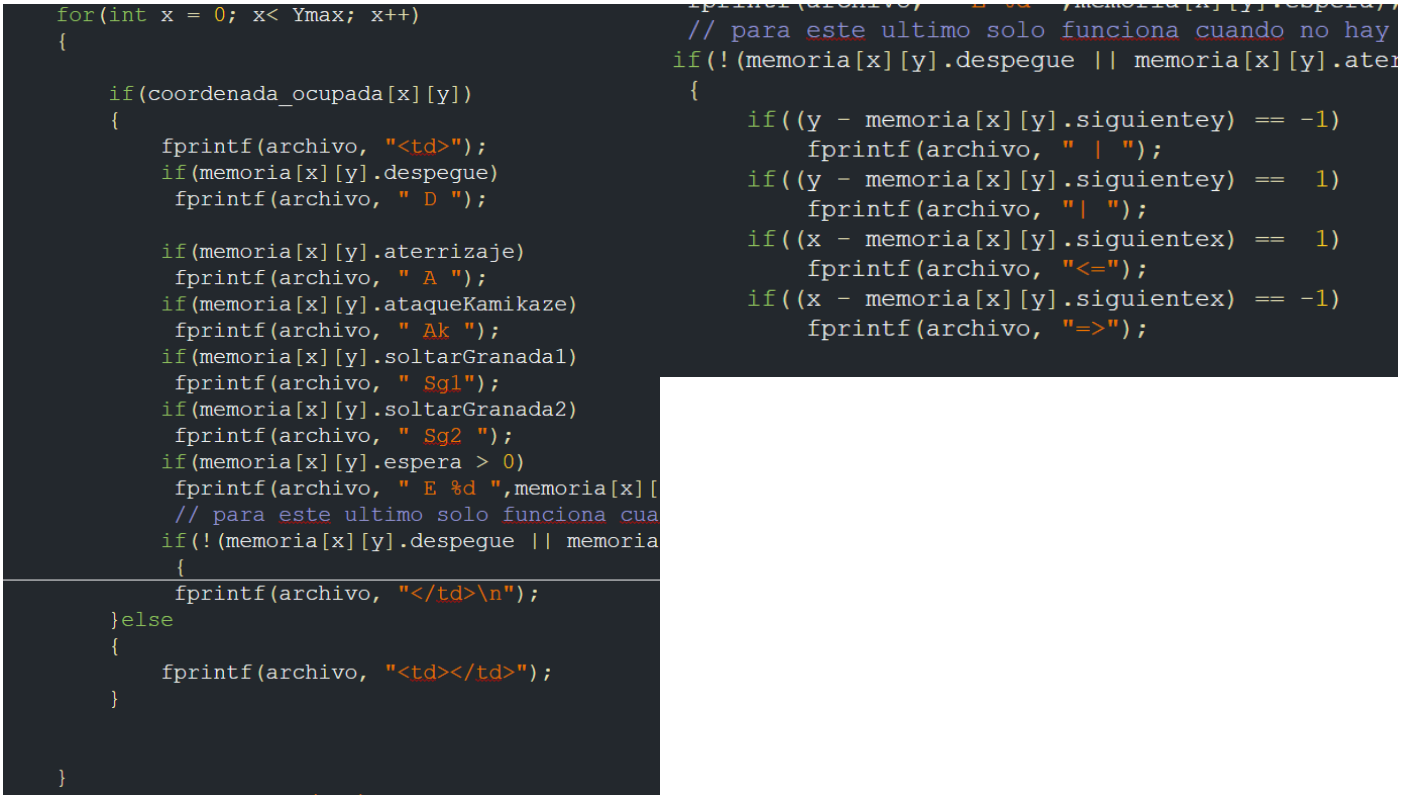
```
fprintf(archivo, "<tr>\n");
fprintf(archivo, "<td>-</td>");
for(int j = 0; j < Xmax; j++)
{
    fprintf(archivo, "<td>%d</td>", j);
}
fprintf(archivo, "</tr>\n");
```

Para el pasaje vertical hago el recorrido de una tabla que son con 2 FOR donde en el primer FOR antes de acceder al otro for creó la fila e introducir el número de coordenadas.

```
for(int y = 0; y < Xmax ; y++)
{
    fprintf(archivo, "<tr>\n");
    fprintf(archivo, "<td>%d</td>\n",y);
    for(int x = 0; x< Ymax; x++)
    {
```

Por último para la colocación de datos de la memoria a archivo html como sabemos que la memoria está limpia de errores a menos que manualmente lo hagamos de manera intencional, mediante IF consultamos por cada dato de la coordenada, se va a insertar de

manera correspondiente, también si sólo hay movimiento se realiza con un IF que todo los datos anteriores están “vacíos”.



```
for(int x = 0; x< Ymax; x++)
{

    if(coordenada_ocupada[x][y])
    {
        fprintf(archivo, "<td>");
        if(memoria[x][y].despegue)
         fprintf(archivo, " D ");

        if(memoria[x][y].aterrizaje)
         fprintf(archivo, " A ");
        if(memoria[x][y].ataqueKamikaze)
         fprintf(archivo, " Ak ");
        if(memoria[x][y].soltarGranada1)
         fprintf(archivo, " Sg1");
        if(memoria[x][y].soltarGranada2)
         fprintf(archivo, " Sg2 ");
        if(memoria[x][y].espera > 0)
         fprintf(archivo, " E %d ",memoria[x][
         // para este ultimo solo funciona cua
        if(!(memoria[x][y].despegue || memoria
         {
         fprintf(archivo, "</td>\n");
    }else
    {
        fprintf(archivo, "<td></td>");
    }


}
```

```
 // para este ultimo solo funciona cuando no hay
if(!(memoria[x][y].despegue || memoria[x][y].ater
 {
     if((y - memoria[x][y].siguientey) == -1)
         fprintf(archivo, " | ");
     if((y - memoria[x][y].siguientey) ==  1)
         fprintf(archivo, "| ");
     if((x - memoria[x][y].siguientex) ==  1)
         fprintf(archivo, "<=");
     if((x - memoria[x][y].siguientex) == -1)
         fprintf(archivo, "=>");
```

USOS DE IA

En este caso he utilizado la IA con claude con respecto a un chequeo del codigo de resultado final

Claude

Solicite que con mi código genere archivos que recorran toda la tabla y también que genere archivos con errores es ahí donde anteriormente en la operación 1 tenía mal cierta lógica ya que anteriormente mi hipótesis es que si el recorrido es correcto no deberían tener coordenadas extras y el recorrido será interpretado si las coordenadas y su coordenadas siguientes son 0, en ese caso no suma, entonces se interpreta que no hay movimiento.
Mi solución era hacerlo más general ignorando esa condición y si hay movimientos extras que no afectan a la trayectoria deseada entonces con la operación 5 y con ver por HTML con la operación 7, sacar esos movimientos que “afectan” a la visibilidad de la trayectoria.
También había otro problema y es que se podía generar un ciclo infinito en la operación 1 cuando se quería analizar las coordenadas mediante for, esto por una trayectoria que era (2,2) => (2,3) => (2,2) esta secuencia verifica ya que es continuo y porque (2,3) tiene como datos en sus movimientos siguientes el (2,2) el bool coord_ocupada [][] no lo detecta como si volviera a pasar por la misma coordenada, entonces plantee un condicional que me determine si se llegó al fin de la trayectoria, esto lo explique en la operación 1.

Acá tuve un problema con **Corregir un registro del archivo,** el puntero al sobreescribir generaba más registros basura haciendo que la operación 1 no pueda cargar en la memoria, la solución fue usar como auxiliar una variable de tipo entero i=0 el cual al encontrar la coordenada en el archivo,"i" tendrá la posición del puntero y con fflush(); fuerzo el pasaje de datos al archivo esto fue con la sugerencia de chatgpt.

Por último anteriormente note que habia un problema con el manejo de la memoria, eso lo descubrí cuando pase por primera vez un prototipo del código que emplee, este link es uno reciente el otro lo emplee con CHATGPT pero sin cuenta por lo que no tengo registro de eso , mi solución era emplear que la memoria del dron y de coordenada ocupada sean de tipo globales, teniendo eso el código aun así funciona sin problemas por otra parte claude me sugiere que use static pero al no saber bien cómo funciona decidí dejarlo como globales. También me corrigió algunos errores que no vi.
https://claude.ai/share/d4d82943-4f4a-47ed-9ebd-65afa95d524c
quedando así

```
34  // por sugerencia de la IA paso las memoria importantes como globales
35     Orden memoria[Xmax][Ymax];
36     bool coordenada_ocupada[Xmax][Ymax] = {false};
37     Orden memoria_temporal[Xmax][Ymax];
38     bool coord_ocupada[Xmax][Ymax] = {false};
39  bool Cargar_Archivo(Orden memoria[Xmax][Ymax],bool coordenada_ocupada[Xmax][Ymax],char r[256])
40  {
```

Los 2 últimos son para la operación 1.

CASOS DE PRUEBA

Para esto hice una trayectoria no tan larga para que primero lea si es correcto
la trayectoria es (2,2) (2,3) (2,4) (3,4) (4,4)
el cual (2,2) es el despegue y (4,4) aterrizaje.
Cómo lo voy a volver hacer entonces creo un archivo y voy ingresando dato por dato.

```
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
Ingrese la direccion del archivo a crear
ArchivoNuevo
Ingrese coordenadas (x,y)
2
2
===DATOS===

Depegue              ==> 0
Aterrizaje           ==> 0
Soltar granada       ==> 0
Soltar granada 2     ==> 0
Ataque Kamikaze      ==> 0
Esperar              ==> 0
Siguente coordenadas ==> (0,0)
Presiona cualquier tecla para continuar
```

```
COORDENADA ACUTAL      ==> (4,4)
Que desea cambiar
Depegue              (1)
Aterrizaje           (2)
Soltar granada       (3)
Soltar granada 2     (4)
Ataque Kamikaze      (5)
Esperar              (6)
Siguente coordenadas (7)
Finalizar            (8)
Aterrizaje ==> True (T) o False (F)
===DATOS===

Depegue              ==> 0
Aterrizaje           ==> 1
Soltar granada       ==> 0
Soltar granada 2     ==> 0
Ataque Kamikaze      ==> 0
Esperar              ==> 0
Siguente coordenadas ==> (0,0)
Presiona cualquier tecla para continuar

COORDENADA ACUTAL      ==> (4,4)
Que desea cambiar
Depegue              (1)
```

Luego ejecutó la operación 1, mi ruta que cree es ArchivoNuevo y con html lo visualizo

```
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
Ingrese la direccion de memoria
ArchivoNuevo
===CARGADO===
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
```

-	0	1	2	3	4	5	6	7
0								
1								
2			D					
3			Sg1					
4			=>	Sg2	A			
5								
6								
7								

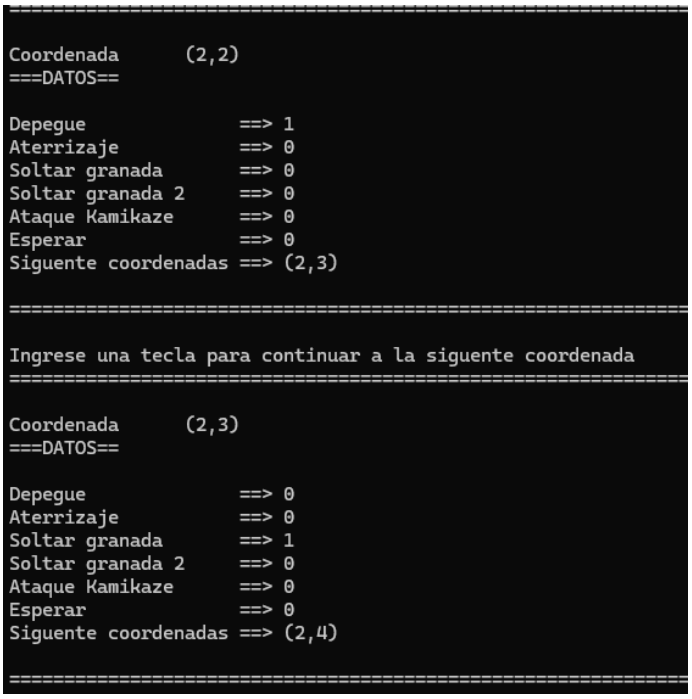
Ahora quiero generar un error, en la coordenada (3,4) voy a soltar granada 1 haciendo que la granada 1 se vuelva a tirar ya que en la coordenada (2,3) se suelta la granada 1. La operación 1 debe indicar ese error

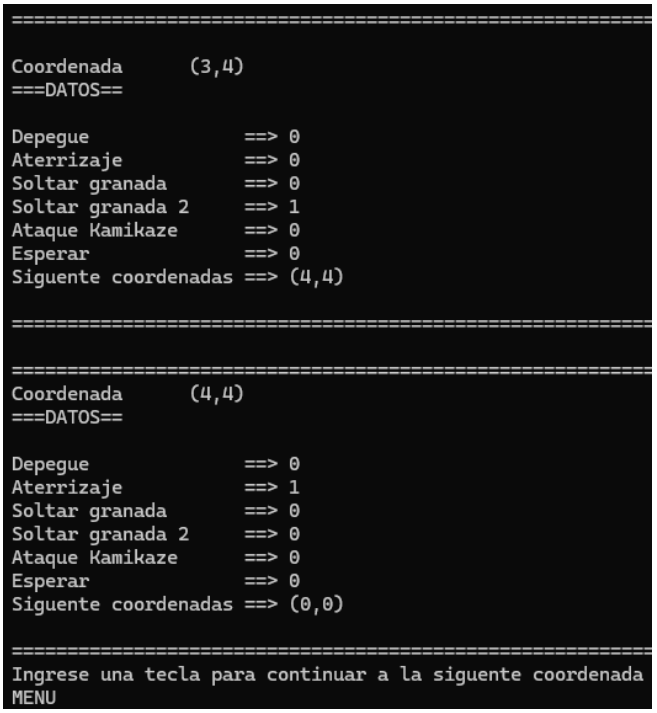
```
COORDENADA ACUTAL      ==> (3,4)
Que desea cambiar
Depegue              (1)
Aterrizaje           (2)
Soltar granada       (3)
Soltar granada 2     (4)
Ataque Kamikaze      (5)
Esperar              (6)
Siguente coordenadas (7)
modificar x e y      (8)
Finalizar            (9)
Soltar Granada 1 ==> True (T) o False (F)
===DATOS===

Depegue              ==> 0
```

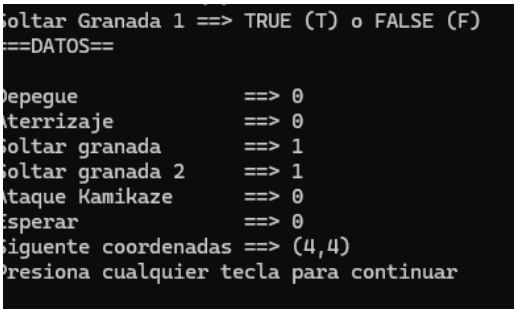
```
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
Ingrese la direccion de memoria
ArchivoNuevo
===ERROR GRANADA 1===
```

Por ahora hice pruebas con las operaciones 3) cree el archivo, 1) se cargó con éxito 4) corregi el archivo con tal de volver a 1) y comprobar de que detecto un error y 7) lo visualice en un html.
Ahora quiero ver el 2) archivo cargado para ver si me muestra acorde de inicio (2,2) a final (4,4)





Como el dron tiene datos en la memoria voy a modificar nuevamente soltar granada 2 de la coordenada (3,4) y poner en true también soltar granada 1



Ahora lo visualizo en html

-	0	1	2	3	4	5	6	7
0								
1								
2					D			
3					|			
4					=>	Sg1 Sg2	A	
5								

En este caso lo muestra correctamente aunque queda claro mencionar que esta permitido también, si corrijo esto en el archivo, la operación 1 lo aceptara, esto es porque tiene sentido soltar 2 granadas en una posición en mi parecer, tambien si quiero finalizar con un ataque kamikaze puedo acabar soltando las granadas para un mayor daño.

Ahora quiero encontrar errores de no continuidad , diagonales y si no hay un final en la trayectoria.
Para ello a (2,3) corto lazo con (2,4) para no continuidad.

```
COORDENADA ACUTAL      ==> (2,3)
Que desea cambiar
Depegue              (1)
Aterrizaje           (2)
Soltar granada       (3)
Soltar granada 2     (4)
Ataque Kamikaze      (5)
Esperar              (6)
Siguente coordenadas (7)
modificar x e y      (8)
Finalizar            (9)
Coordenadas siguentes (x,y)
Coordenada actual (2,3)
0
0
===DATOS===

Depegue              ==> 0
Aterrizaje           ==> 0
Soltar granada       ==> 0
Soltar granada 2     ==> 0
Ataque Kamikaze      ==> 0
Esperar              ==> 0
Siguente coordenadas ==> (0,0)
Presiona cualquier tecla para continuar
```

```
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
Ingrese la direccion de memoria
ArchivoNuevo
Error coordenada (x,y) ==> (2,3)
Movimiento no continuo
En caso de no encontrar las coordenadas pruebe con la coordenada anterior (2,2)
MENU
```

Luego quiero ver si (2,3) pasa a (3,4) que es un movimiento diagonal

```
COORDENADA ACUTAL      ==> (2,3)
Que desea cambiar
Depegue              (1)
Aterrizaje           (2)
Soltar granada       (3)
Soltar granada 2     (4)
Ataque Kamikaze      (5)
Esperar              (6)
Siguente coordenadas (7)
modificar x e y      (8)
Finalizar            (9)
Coordenadas siguentes (x,y)
Coordenada actual (2,3)
3
4
===DATOS===

Depegue              ==> 0
Aterrizaje           ==> 0
Soltar granada       ==> 0
Soltar granada 2     ==> 0
Ataque Kamikaze      ==> 0
Esperar              ==> 0
Siguente coordenadas ==> (3,4)
Presiona cualquier tecla para continuar
```

```
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
Ingrese la direccion de memoria
ArchivoNuevo
Error coordenada (x,y) ==> (2,3)
Movimiento diagonal
En caso de no encontrar las coordenadas pruebe con la coordenada anterior (2,2)
MENU
```

Para terminar, a (4,4) le cambio aterrizaje = false.

```
COORDENADA ACUTAL      ==> (4,4)
Que desea cambiar
Depegue              (1)
Aterrizaje           (2)
Soltar granada       (3)
Soltar granada 2     (4)
Ataque Kamikaze      (5)
Esperar              (6)
Siguente coordenadas (7)
modificar x e y      (8)
Finalizar            (9)
Aterrizaje ==> True (T) o False (F)
===DATOS===

Depegue              ==> 0
Aterrizaje           ==> 0
Soltar granada       ==> 0
Soltar granada 2     ==> 0
Ataque Kamikaze      ==> 0
Esperar              ==> 0
Siguente coordenadas ==> (0,0)
Presiona cualquier tecla para continuar
```

```
Ingrese la direccion de memoria
ArchivoNuevo
---ERROR CON ATAQUE KAMIKAZE O ATERRIZAJE---
El recorrido no tiene finalizacion
COORDENADAS LEIADAS nota: No esta en orden
 (2,2) (2,3) (2,4) (3,4) (4,4)
```

Por último quiero guardar la memoria del dron a un archivo nuevo, la llamaré CreandoArchivo, luego verifico con la operación 1.

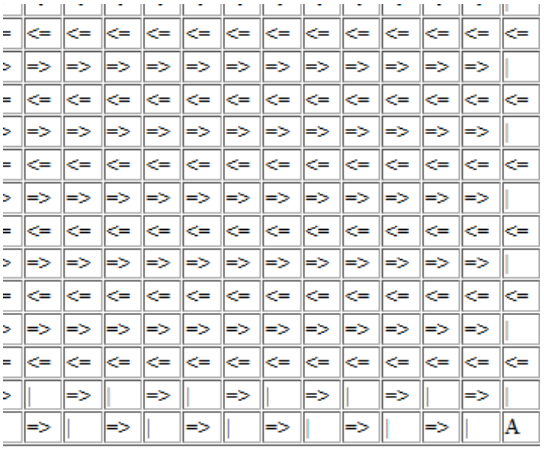
```
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
Ingrese la direccion de memoria
CreandoArchivo
===CARGADO===
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
```

```
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
Ingrese nombre de ruta para el archivo
CreandoArchivo
===HECHO===
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
```

Mis pruebas siempre fue con estas coordenadas puede detectar errores ya sean movimientos diagonales, no continuos, ciclos.
Es por ello que como mencione en USOS DE IA , le pedí códigos para corroborar, Una de ellas ese que recorra toda la tabla para ver el problema del stack overflow, así que copio la dirección y si funciona debería mostrar en el html.

```
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
Ingrese la direccion de memoria
C://Users//Miguel//Downloads//ataque_39999_celdas_fin_199_199.dat
===CARGADO===
```

```
===CARGADO===
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
===HECHO===
```



También le solicite un archivo de errores así que tome uno y con intuición lo fui arreglando algo a mencionar, si bien esta corrección es a ciegas, a veces quizás por un archivo externo exista una trayectoria ambigua pero funcional, en

```
C:\UTN\ALGORITMOS\2026\t
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
Ingrese la direccion de memoria
C://Users//Miguel//Downloads//errores_2_recorrido.dat
Error coordenadas (x,y) ==> (20,30)
Hubo ataque en el despegue
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
MENU
Cargar Archivo De Ataque En Memoria     (1)
```

```
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
Ingrese la direccion del archivo a correir
C://Users//Miguel//Downloads//errores_2_recorrido.dat
Ingrese coordenadas (x,y) ==> 20
30
===DATOS===

Depegue              ==> 1
Aterrizaje           ==> 0
Soltar granada       ==> 1
Soltar granada 2     ==> 0
Ataque Kamikaze      ==> 0
Esperar              ==> 0
Siguente coordenadas ==> (21,30)
Presiona cualquier tecla para continuar
```

ese caso mi consideración en este programa busca que se sepa por parte del usuario la trayectoria exacta.

```
COORDENADA ACUTAL      ==> (20,30)
Que desea cambiar
Depegue               (1)
Aterrizaje            (2)
Soltar granada        (3)
Soltar granada 2      (4)
Ataque Kamikaze       (5)
Esperar               (6)
Siguente coordenadas (7)
modificar x e y       (8)
Finalizar             (9)
Soltar Granada 1 ==> True (T) o False (F)
===DATOS===

Depegue               ==> 1
Aterrizaje            ==> 0
Soltar granada        ==> 0
Soltar granada 2      ==> 0
Ataque Kamikaze       ==> 0
Esperar               ==> 0
Siguente coordenadas ==> (21,30)
Presiona cualquier tecla para continuar
```

```
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
Ingrese la direccion de memoria
C://Users//Miguel//Downloads//ataque_39999_celdas_fin_199_199.dat
Error coordenada (x,y) ==> (22,30)
Movimiento diagonal
En caso de no encontrar las coordenadas pruebe con la coordenada anterior (21,30)
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
```

En este caso hubo un problema de ataque con despegue, movimiento diagonal, movimiento no continuo y al querer finalizar exista movimientos siguientes luego lo visualice en el html

```
SI (S) o NO (N)
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
Ingrese la direccion de memoria
C://Users//Miguel//Downloads//errores_2_recorrido.dat
Error coordenadas(x,y)==> (22,34)
Al intentar finalizar el dato de la cooordenada marca una coordenada siguente mayor a (0,0)
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
```

```
Ataque Kamikaze       (5)
Esperar               (6)
Siguente coordenadas (7)
modificar x e y       (8)
Finalizar             (9)
Buscar otras coordenadas?
SI (S) o NO (N)
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
Ingrese la direccion de memoria
C://Users//Miguel//Downloads//errores_2_recorrido.dat
===CARGADO===
MENU
Cargar Archivo De Ataque En Memoria     (1)
Mostrar Ataque Cargado                  (2)
Crear Un Archivo De Ataque Nuevo        (3)
Corregir Un Registro Del Archivo        (4)
Corregir Un Registro En Memoria         (5)
Guardar Memoria En Un Archivo Nuevo     (6)
Visualizar Un Archivo De Ataque En HTML (7)
Salir                                   (8)
```

| | | |

Conclusión

La resolución de estas operaciones fueron dadas con ciertos criterios que fui viendo a lo largo del progreso el cual hizo realizar cambios como por ejemplo en la operación 1 ya que anteriormente consideraba que la lectura sea de archivos ordenados, pero al hacer la operación 3 y 4 me di cuenta que era más funcional si la lectura fuese tanto con archivos ordenados como con archivos desordenados.
Si bien este trabajo práctico lo realice solo, considero que este programa es funcional después de todo con ciertas fallas que se fue mencionado el cual considero que si tuviera grupo se podría agilizar más una solución mejor.
Por último con los testeos que hice bajo mi pensamiento en cómo debería ser su uso y con medidas de cuidado, se logró llegar a los objetivos que plantea el trabajo práctico permitiendo reforzar temas que vimos anteriormente en clase aunque si quisiera volver a hacerlo , aplicaría conceptos vistos en clases posteriores como punteros, listas DE para tener otro punto de vista a como optimizar mejor el manejo de resoluciones en cada operación.